University of Stirling

ITNPBD2 Representing and Manipulating Data

Assignment Autumn 2025

A Consultancy Job for JC Penney

This notebook forms the assignment instructions and submission document of the assignment for ITNPBD2. Read the instructions carefully and enter code into the cells as indicated.

You will need these five files, which were in the Zip file you downloaded from the course webpage:

- jcpenney_reviewers.json
- jcpenney_products.json
- products.csv
- reviews.csv
- users.csv

The data in these files describes products that have been sold by the American retail giant, JC Penney, and reviews by customers who bought them. Note that the product data is real, but the customer data is synthetic.

Your job is to process the data, as requested in the instructions in the markdown cells in this notebook.

Completing the Assignment

Rename this file to be xxxxxx_BD2 where xxxxxx is your student number, then type your code and narrative description into the boxes provided. Add as many code and markdown cells as you need. The cells should contain:

- **Text narrative describing what you did with the data**
- **The code that performs the task you have described**
- **Comments that explain your code**

The final structure (in PDF) of your report must:

- **Start from the main insights observed (max 5 pages)**
- **Include as an appendix the source code used for producing those insights (max 15 pages)**
- **Include an AI cover sheet (provided on Canvas), which must contain a link to a versioned notebook file in OneDrive or another platform for version checks.**

Marking Scheme

The assessment will be marked against the university Common Marking Scheme (CMS)

Here is a summary of what you need to achieve to gain a grade in the major grade bands:

Grade	Requirement
Fail	You will fail if your code does not run or does not achieve even the basics of the task. You may also fail if you submit code without either comments or a text explanation of what the code does.
Pass	To pass, you must submit sufficient working code to show that you have mastered the basics of the task, even if not everything works completely. You must include some justifications for your choice of methods, but without mentioning alternatives.
Merit	For a merit, your code must be mostly correct, with only small problems or parts missing, and your comments must be useful rather than simply re-stating the code in English. Most choices for methods and structures should be explained and alternatives mentioned.
Distinction	For a distinction, your code must be working, correct, and well commented and shows an appreciation of style, efficiency and reliability. All choices for methods and structures are concisely justified and alternatives are given well thought considerations. For a distinction, your work should be good enough to present to executives at the company.

The full details of the CMS can be found here

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/academic-policy-and-practice/quality-handbook/assessment-policy-and-procedure/appendix-2-postgraduate-common-marking-scheme/

Note that this means there are not certain numbers of marks allocated to each stage of the assignment. Your grade will reflect how well your solutions and comments demonstrate that you have achieved the learning outcomes of the task.

Submission

When you are ready to submit, **print** your notebook as PDF (go to File -> Print Preview) in the Jupyter menu. Make sure you have run all the cells and that their output is displayed. Any lines of code or comments that are not visible in the pdf should be broken across several lines. You can then submit the file online.

Late penalties will apply at a rate of three marks per day, up to a maximum of 7 days. After 7 days you will be given a mark of 0. Extensions will be considered under acceptable circumstances outside your control.

Academic Integrity

This is an individual assignment, and so all submitted work must be fully your own work.

The University of Stirling is committed to protecting the quality and standards of its awards. Consequently, the University seeks to promote and nurture academic integrity, support staff academic integrity, and support students to understand and develop good academic skills that facilitate academic integrity.

In addition, the University deals decisively with all forms of Academic Misconduct.

Where a student does not act with academic integrity, their work or behaviour may demonstrate Poor Academic Practice or it may represent Academic Misconduct.

Poor Academic Practice

Poor Academic Practice is defined as: "The submission of any type of assessment with a lack of referencing or inadequate referencing which does not effectively acknowledge the origin of words, ideas, images, tables, diagrams, maps, code, sound and any other sources used in the assessment."

Academic Misconduct

Academic Misconduct is defined as: "any act or attempted act that does not demonstrate academic integrity and that may result in creating an unfair academic advantage for you or another person, or an academic disadvantage for any other member or member of the academic community."

Plagiarism is presenting somebody else's work as your own **and includes the use of artificial intelligence tools beyond AIAS Level 2 or the use of Large Language Models.**. Plagiarism is a form of academic misconduct and is taken very seriously by the University. Students found to have plagiarised work can have marks deducted and, in serious cases, even be expelled from the University. Do not submit any work that is not entirely your own. Do not collaborate with or get help from anybody else with this assignment.

The University of Stirling's full policy on Academic Integrity can be found at:

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/academic-policy-and-practice/quality-handbook/academic-integrity-policy-and-academic-misconduct-procedure/

The Assignment

Your task with this assignment is to use the data provided to demonstrate your Python data manipulation skills.

There are three `.csv` files and two `.json` files so you can process different types of data. The files also contain unstructured data in the form of natural language in English and links to images that you can access from the JC Penney website (use the field called `product_image_urls`).

Start with easy tasks to show you can read in a file, create some variables and data structures, and manipulate their contents. Then move onto something more interesting.

Look at the data that we provided with this assessment and think of something interesting to do with it using whatever libraries you like. Describe what you decide to do with the data and why it might be interesting or useful to the company to do it.

You can add additional data if you need to - either download it or access it using `requests`. Produce working code to implement your ideas in as many cells as you need below. There is no single right answer, the aim is to simply show you are competent in using python for data analysis. Exactly how you do that is up to you.

For a distinction class grade, this must show originality, creative thinking, and insights beyond what you've been taught directly on the module.

Structure

You may structure the appendix of the project how you wish, but here is a suggested guideline to help you organise your work, based on the CRISP-DM data science methodology:

1. **Business understanding** - What business context is the data coming from? What insights would be valuable in that context, and what data would be required for that purporse?
2. **Data understanding and preparation** - Explore the data and show you understand its structure and relations, with the aid of appropriate visualisation techniques. Assess the data quality, which insights you would be able to answer from it, and what preparation the data would require. Add new data from another source if required to bring new insights to the data you already have.

3. **Data modeling (optional)** - Would modeling be required for the insights you have considered? Use appropriate techniques, if so.
4. **Evaluation and deployment** - How do the insights you obtained help the company, and how can should they be adopted in their business? If modeling techniques have been adopted, are their use scientifically sound and how should they be mantained?

Remember to make sure you are working completely on your own.

Don't work in a group or with a friend

In [143…

```
!pip install pandas numpy seaborn matplotlib scipy --quiet
!pip install nltk --quiet

# Import Libraries
import pandas as pd
import numpy as np
import seaborn as sns
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt
from scipy.stats import pearsonr, spearmanr
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
nltk.download("vader_lexicon", quiet=True)
sia = SentimentIntensityAnalyzer()
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

Step 1: Data Loading and Initial Inspection

The first section of this code will include loading the data and verifying the structure for future use.

In this step:

- Two json files and the three csv files were loaded.
- A dictionary was used to manage the csvs effectively.
- The record counts were printed.
- The first ten rows of all data were then printed to gain an understanding of structure and content.
- This step ensures all data is loaded and visible and some intial information on the dataset is ascertained.

In [144…

```
# Load in the JSON files
products_json_df = pd.read_json("jcpenney_products.json", lines=True)
reviewers_json_df = pd.read_json("jcpenney_reviewers.json", lines=True)
```

```
print(f"Products JSON: {len(products_json_df)} rows | Reviewers JSON: {len(reviewer

# Load in the CSV files
csv_files = {n: pd.read_csv(f"{n.lower()}.csv") for n in ["Products", "Reviews", "U
for n, df in csv_files.items():
    print(f"{n} CSV: {len(df)} rows"); display(df.head(4))

# Unpack dataframes for later use
products_csv_df, reviews_csv_df, users_csv_df = csv_files.values()
```

```
Products JSON: 7982 rows | Reviewers JSON: 5000 rows
Products CSV: 7982 rows
```

	Uniq_id	SKU	Name	Description	Price	Av_Score
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	2.625
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.000
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	2.625
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.500

```
Reviews CSV: 39063 rows
```

	Uniq_id	Username	Score	Review
0	b6c0b6bea69c722939585baeac73c13d	fsdv4141	2	You never have to worry about the fit...Alfred...
1	b6c0b6bea69c722939585baeac73c13d	krpz1113	1	Good quality fabric. Perfect fit. Washed very ...
2	b6c0b6bea69c722939585baeac73c13d	mbmg3241	2	I do not normally wear pants or capris that ha...
3	b6c0b6bea69c722939585baeac73c13d	zeqg1222	0	I love these capris! They fit true to size and...

```
Users CSV: 5000 rows
```

	Username	DOB	State
0	bkpn1412	31.07.1983	Oregon
1	gqjs4414	27.07.1998	Massachusetts
2	eehe1434	08.08.1950	Idaho
3	hkxj1334	03.08.1969	Florida

Step 2: Data Understanding and Quality Check

Before cleaning and merging, the datasets are explored to assess the structure and quality.

In this step:

- Column names are reviewed for each dataset.
- Missing values are identified and summarised.
- A visual heatmap highlights patterns of missing data.
- This step ensures that any inconsistencies or incomplete records can be addressed before further analysis.

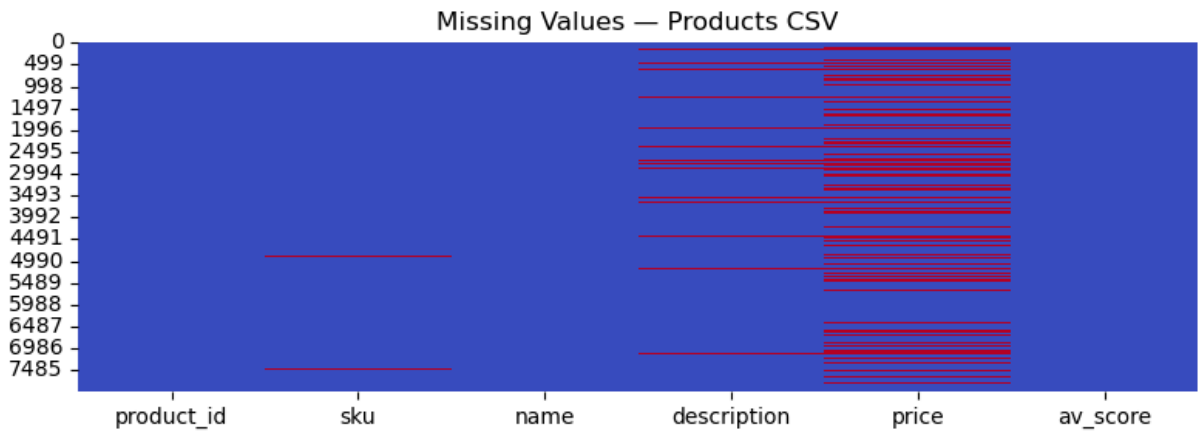
The decision was made to visiualise the data as after it was identified that there were only missing values in one dataset, a heatmap was created to quickly reveal patterns. This allows the ability for clearer, visual interpretation of the comparison of what was missing due to there being over 7,000 products. It also aided with prioritisation of which missingness to address.

In [154…

```
# Step 2: Data Understanding and Quality Check

# Creating a dictionary to store all data framesdatasets =
datasets = {
    "Products CSV": products_csv_df,
    "Reviews CSV": reviews_csv_df,
    "Users CSV": users_csv_df,
    "Products JSON": products_json_df,
    "Reviewers JSON": reviewers_json_df
}

# Summary of structure + top missing values
summary = []
for name, df in datasets.items():
    missing = df.isna().sum().sum()
    summary.append([name, df.shape[0], df.shape[1], missing])

summary_df = pd.DataFrame(summary, columns=["Dataset", "Rows", "Columns", "Total Mi
display(summary_df)

# Visualise missingness only for the dataset with missing data
plt.figure(figsize=(8,3))
sns.heatmap(products_csv_df.isna(), cbar=False, cmap="coolwarm")
```

```
plt.title("Missing Values — Products CSV")
plt.tight_layout()
plt.show()
```

	Dataset	Rows	Columns	Total Missing
0	Products CSV	7982	6	2776
1	Reviews CSV	39063	4	0
2	Users CSV	5000	5	0
3	Products JSON	7982	18	4394
4	Reviewers JSON	5000	4	0



Step 3: Data Cleaning and Preperation After assessing the data structure and quality, the next step is to clean and standardise it to ensure consistency across files before merging.

In this step:

- All collumn names were standardised by converting to lowercase and key fields renamed.
- All missing values were flagged
- DOB was changed into age groups for more effective analysis
- Removed duplicates
- Prepare the data for merging by ensuring join keys were consistant.
- The approach to missing data was to flag the data rather than delete the data. This method was most appropriate as it meant that crucial data was not deleted and the record count was preserved for analysis. It meant that the missing data could be filtered out when needed. This allowed for the comparison of products with and without complete information. The deletion would have resulted in 27% of products being removed due to there being over 2,000 missing prices attached.

The decision was made to group raw age values into age groups, this was because the age values were too granular for meaningful segmentation. Furthermore, the retail industry typically targets broad age demographics therefore this is more digestive for the audience of

the data. This has also allowed for clearer visualisation and the construction of customer personas.

In [146…

```
# Step 3: Clean & Prepare Data

# Standardise + rename
def clean_cols(df): df.columns = df.columns.str.strip().str.lower(); return df
datasets = [clean_cols(d) for d in [products_csv_df, reviews_csv_df, users_csv_df,
for d in datasets: d.rename(columns={'uniq_id':'product_id','username':'user_id'},
products_csv_df, reviews_csv_df, users_csv_df, products_json_df, reviewers_json_df

# Clean pricing & flag discounts
for c in ['list_price','sale_price']:
    if c in products_json_df: products_json_df[c] = products_json_df[c].astype(str)
dfp = products_json_df
dfp['price_missing_flag'] = dfp[['list_price','sale_price']].isna().any(axis=1).ast
dfp['discount_flag'] = ((dfp['sale_price'] < dfp['list_price']) & dfp['sale_price']
dfp['discount_percent'] = np.where(dfp['discount_flag'], ((dfp['list_price']-dfp['s

# Clean user DOB & derive age
if 'dob' in users_csv_df:
    u = users_csv_df
    u['dob'] = pd.to_datetime(u['dob'], format='%d.%m.%Y', errors='coerce')
    u['age'] = ((pd.Timestamp('today') - u['dob']).dt.days/365.25).round(1)
    u['age_group'] = pd.cut(u['age'], [0,25,40,60,100], labels=['<25','25-40','40-6


#  Summary + sample preview
print(f"Missing price: {dfp['price_missing_flag'].sum()} | Discounts: {dfp['discoun
display(dfp[['product_id','list_price','sale_price','discount_flag','discount_perce
display(u[['user_id','dob','age','age_group']].head())
```

```
Missing price: 2166 | Discounts: 5772 | Users missing DOB: 0
```

	product_id	list_price	sale_price	discount_flag	discount_percent
0	b6c0b6bea69c722939585baeac73c13d	41.09	24.16	1	41.2
1	93e5272c51d8cce02597e3ce67b7ad0a	41.09	24.16	1	41.2
2	013e320f2f2ec0cf5b3ff5418d688528	41.09	24.16	1	41.2
3	505e6633d81f2cb7400c0cfa0394c427	41.09	24.16	1	41.2
4	d969a8542122e1331e304b09f81a83f6	41.09	24.16	1	41.2

	user_id	dob	age	age_group
0	bkpn1412	1983-07-31	42.3	40–60
1	gqjs4414	1998-07-27	27.3	25–40
2	eehe1434	1950-08-08	75.3	60+
3	hkxj1334	1969-08-03	56.3	40–60
4	jjbd1412	2001-07-26	24.3	<25

Step 4: Data Verification and Visual Exploration

After ensuring the datasets were fully cleaned, the next step was to verify their structure and begin exploring key patterns in the data.

In this step:

- Column names for all datasets were printed to confirm consistency after cleaning.
- The eight most common brands were identified and their average sale prices visualised.
- Review, product, and user datasets were merged to link customer location with product pricing.
- Total sales values were calculated for each state and the top ten were plotted to highlight key regional markets.

The barcharts were the data visualisation technique chosen as bar charts are optimal for comparing discrete categories on a continuous measure. For the average sale price by the most popular top eight brands, the vertical orientation creates a clear visual hierarchy which makes it easy to interpret which brands are premium vs value. The differing heights also enable instant regonition of price gaps without reading precise values.

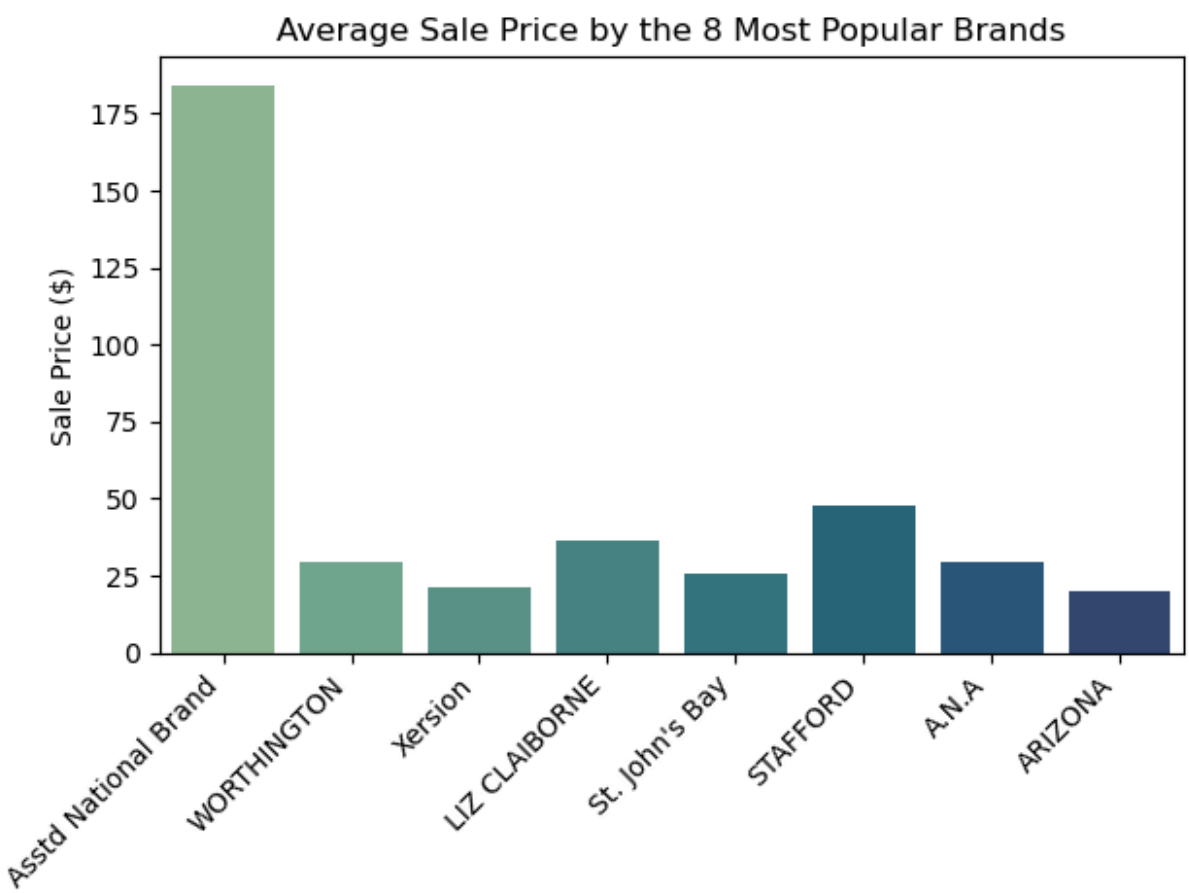
For the top ten locations by estimated sales value bar chart, the horizontal orientation is critical as the location names are longer than the sales numbers and it is imperitive that they are readable. It has also been sorted in descening order to create a natual visual flow. Absoloute dollar values were also used to enable direct budget allocation for JCPenney.

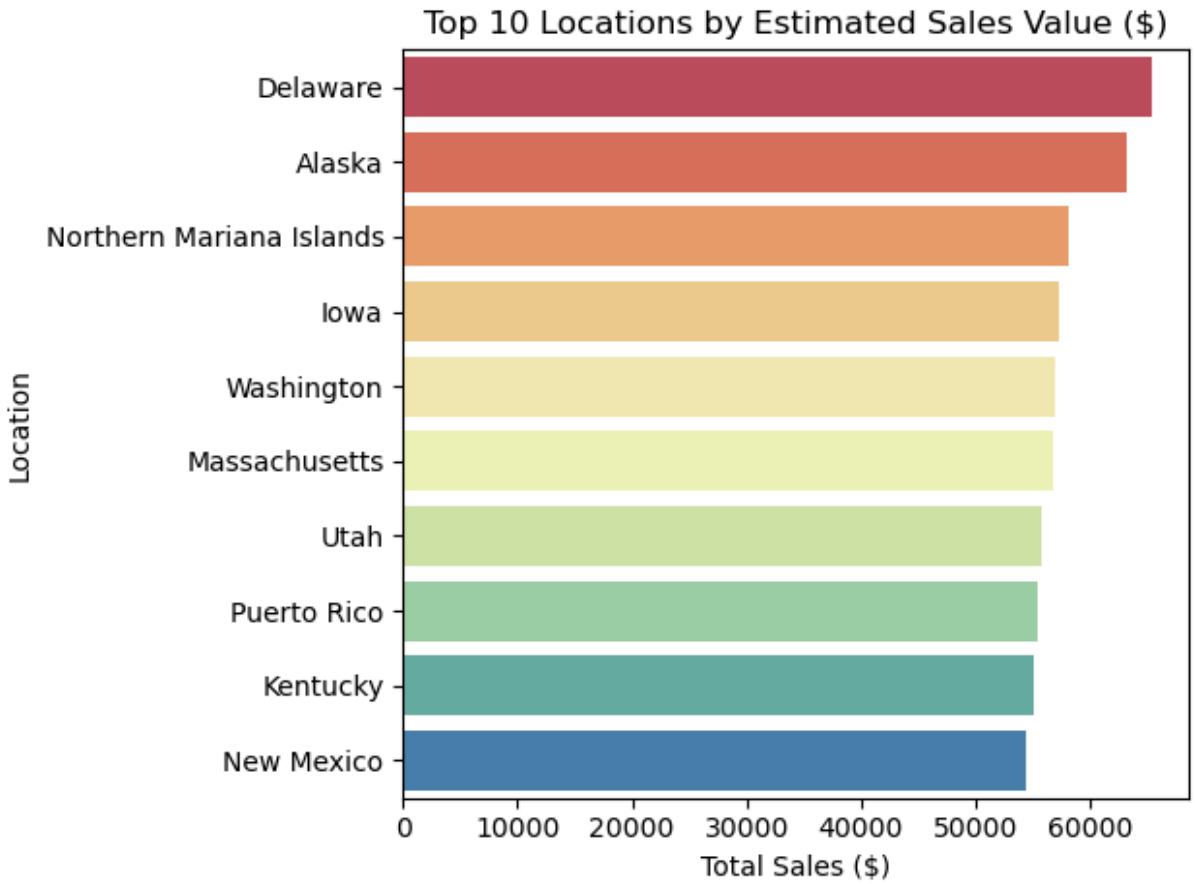
In [147…

```
# Checking all collumn names
for n, d in {"Products":products_csv_df,"Reviews":reviews_csv_df,"Users":users_csv_
    print(f"\n{n}:", list(d.columns))

# Visualisation of average sale price by brand
df = products_json_df.query("price_missing_flag==0 & sale_price.notna() & brand.not
top = df['brand'].value_counts().head(8).index
sns.barplot(df[df['brand'].isin(top)], x='brand', y='sale_price', palette='crest',
plt.title("Average Sale Price by the 8 Most Popular Brands"); plt.xlabel(""); plt.y
plt.xticks(rotation=45, ha='right'); plt.tight_layout(); plt.show()
```

```
# Top 10 states by estimated sales value

# Merge the datasets together to pull the relevent information out
df = (reviews_csv_df
      .merge(reviewers_json_df[['user_id','state']], on='user_id', how='inner')
      .merge(products_json_df[['product_id','sale_price','price_missing_flag']], on
      .query("state.notna() & sale_price.notna() & price_missing_flag == 0"))

# Calculate
sales_by_state = df.groupby('state')['sale_price'].sum().sort_values(ascending=Fals

# Visualise
sns.barplot(x=sales_by_state.values, y=sales_by_state.index,  hue=sales_by_state.in
plt.title("Top 10 Locations by Estimated Sales Value ($)")
plt.xlabel("Total Sales ($)"); plt.ylabel("Location")
plt.tight_layout(); plt.show()
```

```
Products: ['product_id', 'sku', 'name', 'description', 'price', 'av_score']

Reviews: ['product_id', 'user_id', 'score', 'review']

Users: ['user_id', 'dob', 'state', 'age', 'age_group']

ProdJSON: ['product_id', 'sku', 'name_title', 'description', 'list_price', 'sale_pri
ce', 'category', 'category_tree', 'average_product_rating', 'product_url', 'product_
image_urls', 'brand', 'total_number_reviews', 'reviews', 'bought with', 'price_missi
ng_flag', 'discount_flag', 'discount_percent']

RevJSON: ['user_id', 'dob', 'state', 'reviewed']
```





Step 5: Sentiment Processing

With the review text cleaned and prepared, the next step was to apply sentiment analysis and combine this enriched review data with product and user information to build customer personas.

In this step:

- The VADER sentiment model was loaded and applied to each review to generate a compound sentiment score.
- Review data was further cleaned by standardising column names, removing empty entries, and validating rating values.
- The cleaned review dataset was merged with product details and user demographic data to create a unified view of each customer's interactions.

In [148…

```
# Clean review data
df = (reviews_csv_df.copy()
      .rename(columns={'review':'review_text','score':'rating'})
      .assign(review_text=lambda x: x['review_text'].astype(str).str.strip())
      .dropna(subset=['review_text']))

df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
df = df[df['rating'].between(0, 5)]
```

```
# Sentiment scoring
df['sentiment_score'] = df['review_text'].apply(lambda x: sia.polarity_scores(x)['c

print(f"Valid reviews: {len(df):,} | Avg rating: {df['rating'].mean():.2f} | "
      f"Avg sentiment: {df['sentiment_score'].mean():.2f}")
display(df[['user_id','product_id','rating','sentiment_score','review_text']].head(

# Merge reviews → products → users
merged = (df
          .merge(products_json_df, on="product_id", how="left")
          .merge(users_csv_df, on="user_id", how="left"))

print(f"Merged dataset: {len(merged):,} rows")
```

```
Valid reviews: 39,063 | Avg rating: 1.49 | Avg sentiment: 0.67
```

	user_id	product_id	rating	sentiment_score	review_text
0	fsdv4141	b6c0b6bea69c722939585baeac73c13d	2	0.9423	You never have to worry about the fit...Alfred...
1	krpz1113	b6c0b6bea69c722939585baeac73c13d	1	0.8408	Good quality fabric. Perfect fit. Washed very ...
2	mbmg3241	b6c0b6bea69c722939585baeac73c13d	2	0.9514	I do not normally wear pants or capris that ha...
3	zeqg1222	b6c0b6bea69c722939585baeac73c13d	0	0.9329	I love these capris! They fit true to size and...

```
Merged dataset: 39,080 rows
```

Step 6: Sentiment Visualisation and Word Analysis

After constructing the persona dataset, visual analysis was used to explore the relationship between sentiment and product ratings, followed by an examination of the most common terms appearing in customer reviews.

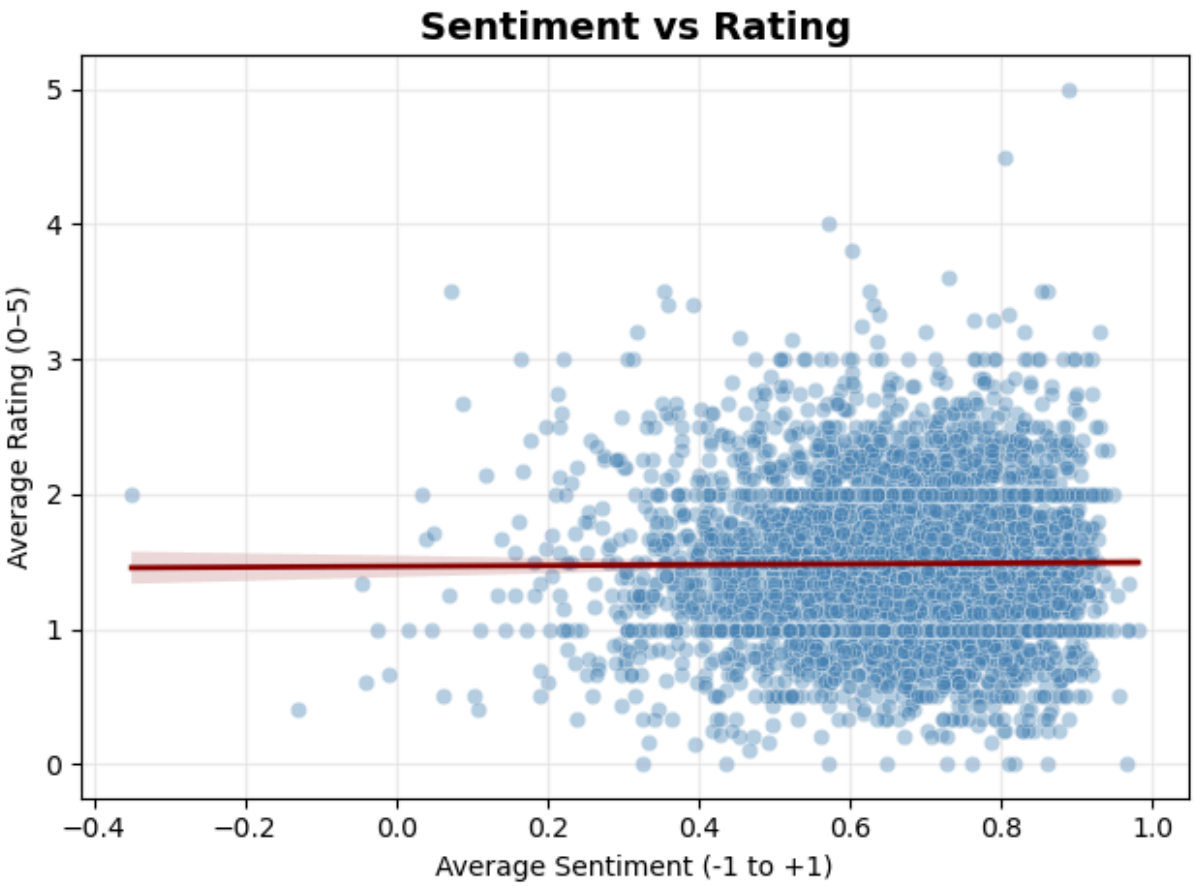
In this step:

- A scatterplot with a regression line was created to show how sentiment scores relate to numerical ratings across users.
- Reviews were grouped into positive, negative, and overall text sets for later exploration.

- Additional stopwords were added to remove generic or uninformative terms commonly found in clothing reviews.
- A wordcloud was generated to highlight the most frequent meaningful words used across all customer reviews.

In [135…

```
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

#  Visualise sentiment vs rating
sns.scatterplot(data=persona_df, x="avg_sentiment", y="avg_rating",
                color="steelblue", alpha=0.4)

sns.regplot(data=persona_df, x="avg_sentiment", y="avg_rating",
            scatter=False, color="darkred", line_kws={"linewidth":2})

plt.title("Sentiment vs Rating", fontsize=14, weight="bold")
plt.xlabel("Average Sentiment (-1 to +1)")
plt.ylabel("Average Rating (0–5)")
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
# Create combined text sets
texts = {
    "All Reviews": df['review_text'],
    "Positive Reviews": df.loc[df['sentiment_score'] > 0.5, 'review_text'],
    "Negative Reviews": df.loc[df['sentiment_score'] < 0, 'review_text']
}

# Wordcloud of the most common words in all reviews

# Removing frequent, non-informative words to improve the quality and clarity of th
stop = STOPWORDS.union({"dress","buy","bought","size","fit","product","good","love"
texts = {
    "Most Common Words in All Reviews": df["review_text"]
}
for title, s in texts.items():
    wc = WordCloud(width=1000, height=600, background_color="white",
                   stopwords=stop, colormap="cool", max_words=150
                  ).generate(" ".join(s.dropna()))
    plt.figure(figsize=(10,6)); plt.imshow(wc); plt.axis("off"); plt.title(title, f
```



Most Common Words in All Reviews
great
comfortable
perfect
look
wear
quality
one
shoe
purchased
shirt
ordered
colors
price
short
pair
work
store
well
jean
fabric
material
soft
still
right
little
true
use
purchase
looking
time
top
find
boot
feel
back
large
waist
wanted
small
easy
happy
loved
way
beautiful
even
make
order
found
put
made
black
style
set
always
length
go
room
much
year
keep
item
fits
wash
son
see
dont
really
got
husband
bra
used
day
summer
jacket
perfectly
enough
around
came
cute
stylish
warm
disappointed
best
recommend
Im
white
now
wearing
tried
take
bit
Thank
thought
better
sale
side
daughter
excellent
hard
brand
lot
return
long
try
loves
without
going
say
will
need
give
pretty
exactly
everything
needed
Ive
received
far
seem
come
leg
run
didnt
first
tall
able
worn
tight
feet
new
online
pleased
thing
rug
looked
well made
want
wish
line
JCP
expected
think
two
light
last
bed
see
dont
not
one
work
shoe
store
pair
black
made
pleased
small
easy
price
loved
short
way
even
beautiful
feel
back
large
waist
wanted
tall
ordered
happy
come
need
pretty
exactly
everything
needed
colors
will
bra
received

In this step:

- Only customers with valid values for rating, sentiment, age, review activity and spend were included, ensuring no imputation or artificial data.
- All features were standardised so each metric contributed equally to the clustering process.
- A stable K-Means model (fixed seeds and multiple initialisations) was used to identify four natural behavioural groups.
- Cluster labels were written back to the full dataset, with incomplete cases left unassigned.
- Cluster centroids were converted back to original units (e.g., actual age and spend) for clear interpretation.

In [149…

```
# Build user-level persona dataset
persona_df = (
    merged.groupby("user_id")
    .agg({
        "rating": "mean",                   # average rating per user
        "sentiment_score": "mean",          # average sentiment per user
        "age": "mean",                      # average age
        "product_id": "count",              # number of reviews/products
        "sale_price": "mean",               # average spend
        "category": lambda x: x.mode().iloc[0] if not x.mode().empty else "unknown"
    })
    .rename(columns={
        "rating": "avg_rating",
        "sentiment_score": "avg_sentiment",
        "age": "avg_age",
        "product_id": "review_count",
        "sale_price": "avg_price",
        "category": "fav_category"
    })
    .reset_index()
)

# Select clustering features
features = ['avg_rating','avg_sentiment','avg_age','review_count','avg_price']

# Keep only customers with complete data (no imputation)
df_clust = persona_df.dropna(subset=features)
print(f"Using {len(df_clust)} customers for clustering")
print(f"{len(persona_df) - len(df_clust)} customers excluded due to missing values\

# Scale features to equalise influence
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_clust[features])

# Run stable K-means clustering
np.random.seed(42)
```

```python
kmeans = KMeans(n_clusters=4, random_state=42, n_init=50, max_iter=500)
labels = kmeans.fit_predict(X_scaled)

# Store cluster labels back in the full persona_df
persona_df['cluster'] = pd.NA
persona_df.loc[df_clust.index, 'cluster'] = labels

print("Customers per cluster:")
print(pd.Series(labels).value_counts().sort_index())

# Convert centroids back to original units (for interpretation)
centroids = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=features
).round(2)
centroids.index.name = "cluster"

print("\nCluster centroids (original units):")
display(centroids)
```

```
Using 4993 customers for clustering
0 customers excluded due to missing values

Customers per cluster:
0    1869
1     291
2    1848
3     985
Name: count, dtype: int64

Cluster centroids (original units):
```

cluster	avg_rating	avg_sentiment	avg_age	review_count	avg_price
0	1.51	0.72	35.89	8.12	77.96
1	1.54	0.66	52.19	7.35	493.81
2	1.48	0.73	63.09	8.24	75.33
3	1.43	0.45	51.86	6.64	80.22

Step 7: Persona Comparison Visualisation

This step visualises how the four data-driven personas differ across the key behavioural metrics used in the segmentation.

In this step:

- Final persona labels are applied to each cluster, replacing numeric IDs with meaningful group names.
- Persona-level averages are calculated to capture the typical behaviour of each group.

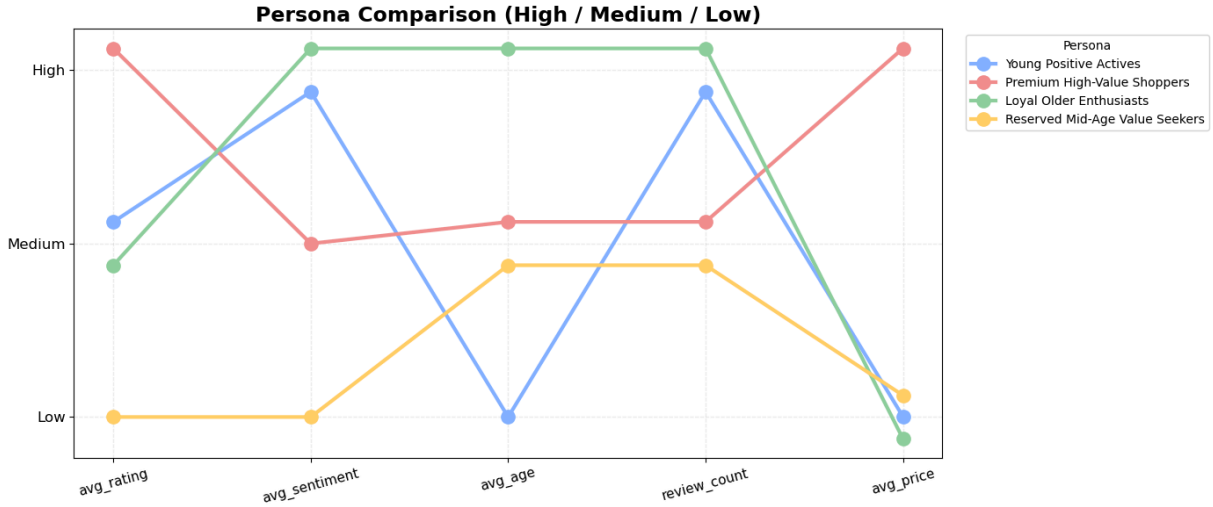
- These averages are converted into Low / Medium / High bands to place all metrics on a shared scale.
- Minor spacing adjustments prevent overlap while maintaining each metric's band category.
- A comparative line chart is produced to show how each persona positions across the behavioural dimensions.

This provides a clear visual summary of the distinct characteristics and relative strengths of each persona.

In [153…

```
# STEP 1 — Assign final persona labels to each user. The cluster model identifies f
# each cluster ID is mapped to its corresponding persona label.

persona_labels = {
    0: "Young Positive Actives",
    1: "Premium High-Value Shoppers",
    2: "Loyal Older Enthusiasts",
    3: "Reserved Mid-Age Value Seekers"
}

persona_df["persona_name"] = persona_df["cluster"].map(persona_labels)

# STEP 2 — Define the personas, metrics, and colours used in the comparison visuali
# we want to compare across personas.

personas = list(persona_labels.values())
metrics = ["avg_rating", "avg_sentiment", "avg_age", "review_count", "avg_price"]

colors = {
    "Young Positive Actives": "#82B2FF",
    "Premium High-Value Shoppers": "#F28E8E",
    "Loyal Older Enthusiasts": "#8FD19E",
    "Reserved Mid-Age Value Seekers": "#FFCE67"
}

x = range(len(metrics))

# STEP 3 — Calculate the average value of each behavioural metric for each persona.
# representing the 'typical' customer in each group.

means = (
    persona_df[persona_df["persona_name"].isin(personas)]
    .groupby("persona_name")[metrics]
    .mean()
)
# STEP 4 — Assign meaningful Low / Medium / High bands using real behavioural thres

cat_df = pd.DataFrame(index=means.index)

banding_rules = {
    "avg_rating":     [0, 1.45, 1.52, 5],
    "avg_sentiment":  [0, 0.55, 0.70, 1],
```

```
    "avg_age":        [0, 40, 55, 120],
    "review_count":   [0, 6, 8, 50],
    "avg_price":      [0, 150, 300, 9999]   # FIX: all £75–80 personas fall into "L
}

for m in metrics:
    bins = banding_rules[m]
    labels = ["Low", "Medium", "High"]
    cat_df[m] = pd.cut(means[m], bins=bins, labels=labels, include_lowest=True)

# Convert categorical bands into numeric positions
band_map = {"Low": 1.0, "Medium": 2.0, "High": 3.0}
band_vals = cat_df.apply(lambda col: col.map(band_map))


for m in metrics:
    vals = means[m]
    for b in [1, 2, 3]:
        mask = band_vals[m] == b
        if mask.sum() > 1:
            scaled = (vals[mask] - vals[mask].min()) / (vals[mask].max() - vals[mas
            micro_scaled.loc[mask, m] = b + (scaled - 0.5) * 0.25
# STEP 6 — Produce the persona comparison visualisation. This plot shows how each p
# on a unified Low–High scale.
plt.figure(figsize=(14,6))

for persona in personas:
    plt.plot(
        x,
        micro_scaled.loc[persona],
        marker="o",
        markersize=11,
        linewidth=3,
        label=persona,
        color=colors[persona]
    )

plt.yticks([1,2,3], ["Low", "Medium", "High"], fontsize=12)
plt.xticks(x, metrics, rotation=15, fontsize=11)
plt.grid(alpha=0.25, linestyle="--")
plt.title("Persona Comparison (High / Medium / Low)", fontsize=17, weight="bold")
plt.legend(title="Persona", bbox_to_anchor=(1.02,1), loc="upper left")

plt.tight_layout()
plt.show()
```



In [160…

```
%%capture
!jupyter nbconvert "2718233_BD2.ipynb" --to webpdf --allow-chromium-download
```